



Smart Contract Security Audit Report



Table Of Contents

1 Executive Summary	_____
2 Audit Methodology	_____
3 Project Overview	_____
3.1 Project Introduction	_____
3.2 Vulnerability Information	_____
4 Code Overview	_____
4.1 Contracts Description	_____
4.2 Visibility Description	_____
4.3 Vulnerability Summary	_____
5 Audit Result	_____
6 Statement	_____

1 Executive Summary

On 2026.08.04, the SlowMist security team received the DMZ Finance team's security audit application for RWAAlpha, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.

Level	Description
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit

Serial Number	Audit Class	Audit Subclass
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

RWAlpha is an on-chain wrapping and subscription/redemption system designed for RWA and fund share scenarios.

The audit scope only covers the LabsWrapperVault and LabsWrappedToken modules.

The system uses the vault contract to receive users' stablecoin subscription requests, interfaces with the official fund share subscription and redemption processes, and mints or burns wrapped tokens proportionally after official assets are received or burn confirmations are completed. This enables the integration between on-chain share circulation and off-chain asset settlement.

Overall, the protocol is not a fully autonomous on-chain system. Instead, it relies on officialFundController, off-chain business systems, and multiple privileged roles to complete critical operations such as subscription confirmation, redemption processing, and fund transfers.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Subscription Orders Do Not Establish Terminal States or Bind to Recipients	Design Logic Audit	Low	Acknowledged
N2	Redeem Settlement Is Not Bound to Queue Records or Recipient Addresses	Design Logic Audit	Low	Acknowledged
N3	Unclassified Stablecoins Can Be Rescued and Withdrawn	Design Logic Audit	Suggestion	Acknowledged
N4	Agent Deposit Is Not Bound to User	Authority Control Vulnerability Audit	Suggestion	Acknowledged

NO	Title	Category	Level	Status
	Authorization Instructions			
N5	Rejected Redemption Lacks State Rollback and Asset Reconciliation Process	Design Logic Audit	Medium	Acknowledged
N6	<code>officialToken</code> Balance Check in <code>confirmOfficialBurnBatch</code> Cannot Effectively Validate Redemption Burn Results	Design Logic Audit	Suggestion	Acknowledged
N7	Stablecoin Configuration Consistency Is Not Validated	Others	Suggestion	Acknowledged
N8	Global refId Can Be Preemptively Occupied Across Different Workflows	Denial of Service Vulnerability	Low	Acknowledged
N9	Withdrawal Requests Only Record Unverified User Input	Design Logic Audit	Suggestion	Acknowledged
N10	Critical Asset Flows Depend on a Centralized Trust Chain	Authority Control Vulnerability Audit	Low	Acknowledged
N11	Official Mint Rejection Lacks a User Refund Path	Design Logic Audit	Low	Acknowledged

4 Code Overview

4.1 Contracts Description

Audit Version:

<https://github.com/dmzFinance/rwalpha-contract>

commit: ffa5cdac1e393c2ae201b3c8095a583e01540d9c

Audit Scope:

```
.
├── LabsWrappedToken.sol
└── LabsWrapperVault.sol
```

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

LabsWrappedToken			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
decimals	Public	-	-
mint	External	Can Modify State	onlyRole
burnFrom	External	Can Modify State	onlyRole
bindVault	External	Can Modify State	onlyRole
grantRole	Public	Can Modify State	onlyRole
_update	Internal	Can Modify State	-

LabsWrapperVault			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
classifiedStableBalance	Public	-	-
unregisteredStableBalance	Public	-	-
classifiedOfficialBacking	Public	-	-
unregisteredOfficialBalance	Public	-	-
backingStatus	External	-	-
depositStable	External	Can Modify State	nonReentrant whenNotPaused onlyRole
requestDeposit	External	Can Modify State	nonReentrant whenNotPaused
depositStableFor	External	Can Modify State	nonReentrant whenNotPaused
requestDepositFor	External	Can Modify State	nonReentrant whenNotPaused onlyRole
_depositStableFrom	Private	Can Modify State	-
submitOfficialMint	External	Can Modify State	nonReentrant whenNotPaused onlyRole
registerOfficialMintSettlement	External	Can Modify State	nonReentrant whenNotPaused onlyRole
registerOfficialMintRejection	External	Can Modify State	nonReentrant whenNotPaused onlyRole
registerOfficialReceipt	External	Can Modify State	nonReentrant whenNotPaused onlyRole
mintAgainstRegistered	External	Can Modify State	nonReentrant whenNotPaused onlyRole

LabsWrapperVault			
burnApprovedAndQueueOfficialBurn	External	Can Modify State	nonReentrant whenNotPaused onlyRole
burnAndRequestOfficialRedeem	External	Can Modify State	nonReentrant whenNotPaused onlyRole
requestRedeem	External	Can Modify State	nonReentrant whenNotPaused
requestRedeemFor	External	Can Modify State	nonReentrant whenNotPaused onlyRole
confirmOfficialBurnBatch	External	Can Modify State	nonReentrant whenNotPaused onlyRole
releaseRedeemStableProceeds	External	Can Modify State	nonReentrant whenNotPaused onlyRole
registerDividendStableReceipt	External	Can Modify State	nonReentrant whenNotPaused onlyRole
releaseDividendStableProceeds	External	Can Modify State	nonReentrant whenNotPaused onlyRole
requestWithdrawal	External	Can Modify State	nonReentrant whenNotPaused
pause	External	Can Modify State	onlyRole
unpause	External	Can Modify State	onlyRole
rescueToken	External	Can Modify State	nonReentrant onlyRole
_burnAndRequestOfficialRedeem	Private	Can Modify State	-
_requireSupplyInvariant	Private	-	-
_consumeRef	Private	Can Modify State	-

4.3 Vulnerability Summary

[N1] [Low] Subscription Orders Do Not Establish Terminal States or Bind to Recipients

Category: Design Logic Audit

Content

In `LabsWrapperVault`, subscription funds are currently managed only through aggregated balances such as `pendingStableDeposits`. Although `stableDepositOrders` records fields including `payer`, `recipient`, and `amount`, subsequent processing flows do not read from or update these orders. The official minting request is not bound to a specific deposit order, rejected requests only return funds to the aggregated pending mint balance, and `mintAgainstRegistered` can mint tokens to an arbitrary `to` address.

As a result, the on-chain state cannot guarantee that a specific deposit will ultimately mint tokens only for the originally designated recipient. It also cannot independently support refunds for subscriptions that are rejected or remain unsettled for an extended period. This issue would require operational role failure, compromise, or an off-chain process error to become an actual fund loss scenario, but user rights cannot be independently verified or enforced by the on-chain logic.

Code location: `LabsWrapperVault.sol#L491`

```
function mintAgainstRegistered(
    address to,
    uint256 amount,
    bytes32 refId
) external nonReentrant whenNotPaused onlyRole(MINT_OPERATOR_ROLE) {
    ...
    require(officialToken.balanceOf(address(this)) >= classifiedOfficialBacking(),
"official_balance_deficit");
    wrappedToken.mint(to, amount);
    ...
}
```

Solution

Unless this is an intentional design choice, each subscription should have an explicit lifecycle state machine, such as: `Created → Submitted → Settled → Minted / Rejected / Cancelled`. The same order ID should be

used throughout the official minting request, settlement, and minting processes. The order record should store the recipient, maximum mintable amount, and already minted amount, and `mintAgainstRegistered` should be restricted to minting only to the recipient specified by the corresponding order.

Status

Acknowledged; After discussion with the project team, the team confirmed that this is an intended design. The protocol relies on centralized roles to ensure consistency throughout the entire process.

[N2] [Low] Redeem Settlement Is Not Bound to Queue Records or Recipient Addresses

Category: Design Logic Audit

Content

In LabsWrapperVault, within `requestRedeem`, `confirmOfficialBurnBatch`, and `releaseRedeemStableProceeds`, the `receiver` field is only recorded in the `RedeemRequested` event and is not persisted into `RedeemQueue`. Although the confirmation stage records `payoutAmount` for each queue item, the payout function only validates the aggregated variable `pendingRedeemStableProceeds`. The current implementation allows `PAYMENT_OPERATOR_ROLE` to specify arbitrary `to` addresses and arbitrary amounts within the accumulated available range. Additionally, there is no “already paid” state tracking mechanism.

As a result, the on-chain logic cannot guarantee that a specific redemption payout is transferred to the original requester or the designated recipient. It also cannot prevent the same queue-related funds from being repeatedly processed through the operational settlement flow. This issue relies on the payment operator performing incorrect operations or being compromised before resulting in an actual fund loss, but the correctness of user redemption rights cannot be independently enforced by the contract.

Code location: LabsWrapperVault.sol#L609

```
function releaseRedeemStableProceeds (
    address to,
    uint256 amount,
    bytes32 refId
) external nonReentrant whenNotPaused onlyRole(PAYMENT_OPERATOR_ROLE) {
```

```
...
    stablecoin.safeTransfer(to, amount);
    emit RedeemStableProceedsReleased(refId, to, amount,
pendingRedeemStableProceeds);
}
```

Solution

Unless this is an intentional design choice, the `RedeemQueue` structure should explicitly store the recipient address, payable amount, paid amount, and final settlement status. The payout interface should be changed to settle based on `queueRefId`, with the recipient address strictly matched against the queue record. Partial payouts should track the remaining payable balance, while fully settled redemptions should transition into an irreversible terminal state to prevent repeated execution.

Status

Acknowledged; After discussion with the project team, the team confirmed that this is an intended design. The protocol relies on centralized roles to ensure consistency throughout the entire process.

[N3] [Suggestion] Unclassified Stablecoins Can Be Rescued and Withdrawn

Category: Design Logic Audit

Content

In the LabsWrapperVault contract, the `rescueToken` function allows administrators to rescue stablecoins within the range of `unregisteredStableBalance`. Dividend payments or redemption proceeds are typically first transferred into the Vault and are then categorized through `registerDividendStableReceipt` or `confirmOfficialBurnBatch`. Before these classification transactions are executed, the received funds are considered “unregistered balances.” As a result, an address holding `DEFAULT_ADMIN_ROLE` could withdraw stablecoins during this time window even if those funds are actually intended for user redemption proceeds or dividend distributions. Subsequent classification operations may then fail due to insufficient balance, causing the protocol’s internal accounting to diverge from the actual asset state.

Code location: LabsWrapperVault.sol#L677-L679

```
function rescueToken(address token, address to, uint256 amount) external
nonReentrant onlyRole(DEFAULT_ADMIN_ROLE) {
    ...
    if (token == address(stablecoin)) {
        require(amount <= unregisteredStableBalance(),
"cannot_rescue_pending_stable");
    }
    IERC20(token).safeTransfer(to, amount);
    emit TokenRescued(token, to, amount);
}
```

Solution

The rescue mechanism should be protected with additional safeguards. It is recommended to introduce delayed execution and multisignature approval requirements, and only allow rescue operations for balances that have passed an explicit grace period and are not associated with any pending business processes. This ensures that operational rescue functionality cannot interfere with legitimate user funds awaiting registration, settlement, or distribution.

Status

Acknowledged

[N4] [Suggestion] Agent Deposit Is Not Bound to User Authorization Instructions

Category: Authority Control Vulnerability Audit

Content

In the LabsWrapperVault contract, within the `depositStableFor` and similar `requestDepositFor` flows, `OFFICIAL_MINT_OPERATOR_ROLE` can independently specify the `payer`, `recipient`, `amount`, and `refId` as long as the user has previously granted ERC20 allowance to the Vault. However, the current implementation does not verify that the user has provided a signed authorization for the specific deposit order.

As a result, an operator with `OFFICIAL_MINT_OPERATOR_ROLE` can initiate deposit operations using a user's existing allowance without proving that the user approved the specific recipient, amount, or order parameters. This weakens the binding between the user's token authorization and the actual deposit instruction, meaning the on-chain logic cannot independently verify that the deposit operation reflects the user's intended action.

Code location: LabsWrapperVault.sol#L315-L335,L498-L515,L531

```
function depositStableFor(
    address payer,
    address recipient,
    uint256 amount,
    bytes32 refId
) external nonReentrant whenNotPaused {
    ...
}

function requestDepositFor(
    address owner,
    address receiver,
    uint256 assets,
    bytes32 requestId
) external nonReentrant whenNotPaused onlyRole(OFFICIAL_MINT_OPERATOR_ROLE) {
    ...
}

function burnApprovedAndQueueOfficialBurn(
    address holder,
    uint256 amount,
    bytes32 refId
) external nonReentrant whenNotPaused onlyRole(REDEEM_OPERATOR_ROLE) {
    ...
}

function burnAndRequestOfficialRedeem(
    address holder,
    uint256 amount,
    bytes32 refId
) external nonReentrant whenNotPaused onlyRole(REDEEM_OPERATOR_ROLE) returns
(uint256 officialBurnRequestId) {
    ...
}

function requestRedeemFor(
    address owner,
    uint256 shares,
    address receiver,
    bytes32 requestId
) external nonReentrant whenNotPaused onlyRole(REDEEM_OPERATOR_ROLE) returns
```

```
(uint256 officialBurnRequestId) {  
    ...  
}
```

Solution

Strengthen the authorization model for delegated deposits. Each delegated deposit should require a user-signed order containing the relevant parameters, such as `payer`, `recipient`, `amount`, `refId`, and a nonce or expiration timestamp. Additionally, users should avoid granting unlimited ERC20 allowances to the Vault whenever possible. Using limited allowances reduces the potential impact of compromised operator roles or unexpected execution paths.

Status

Acknowledged; After discussion with the project team, the team confirmed that this is an intended design.

[N5] [Medium] Rejected Redemption Lacks State Rollback and Asset Reconciliation Process

Category: Design Logic Audit

Content

In the LabsWrapperVault contract, the `confirmOfficialBurnBatch` function is used to handle cases where the official redemption burn has been successfully settled. However, the contract does not provide a corresponding “official redemption rejection” handling entry point similar to `registerOfficialMintRejection`.

If the FundController rejects the burn request and returns the `officialToken` back to the contract, the corresponding `RedeemQueue` will remain in the `Queued` state. Although privileged roles can restore the user's burned shares through `registerOfficialReceipt` and `mintAgainstRegistered`, internal accounting variables such as `pendingOfficialBurn` cannot be rolled back, resulting in an incomplete redemption state machine.

This may cause inconsistencies between the redemption queue status, internal accounting records, and actual asset state. Although the issue requires the trusted operational process to encounter an error or fail to properly handle the rejection flow before causing an impact, the current on-chain logic does not provide a complete recovery path for rejected redemptions.

Solution

A dedicated official redemption rejection handling function should be introduced. A trusted role should invoke this function only after verifying the off-chain rejection result and confirming that the returned assets have been received. The function should perform terminal state processing for the specified `RedeemQueue` and update all related internal accounting accordingly, including rolling back pending burn records and restoring the redemption lifecycle to a consistent state.

Status

Acknowledged; After discussion with the project team, the team confirmed that although `FundController` provides a rejection mechanism, in the actual business process it will not reject requests submitted by the Vault.

[N6] [Suggestion] `officialToken` Balance Check in `confirmOfficialBurnBatch` Cannot Effectively Validate Redemption Burn Results

Category: Design Logic Audit

Content

In the `LabsWrapperVault` contract, the `confirmOfficialBurnBatch` function is responsible for confirming that a batch of redemption requests has been successfully burned by the official system and recording the corresponding stablecoin settlement amounts. At the end of the function, the contract performs an `officialToken` balance check through `classifiedOfficialBacking`. However, this function itself does not directly modify the `officialToken` balance. The balance change caused by the burn request has already been handled within `_burnAndRequestOfficialRedeem`.

Therefore, this balance check cannot provide an effective validation of whether the current confirmation operation actually corresponds to a successful official burn result. The check does not establish any additional security guarantee for the redemption confirmation process and is effectively a redundant validation.

Code location: `LabsWrapperVault.sol#L583`

```
function confirmOfficialBurnBatch(
    bytes32 batchRefId,
    bytes32[] calldata queueRefIds,
    bytes32 evidenceHash
) external nonReentrant whenNotPaused onlyRole(OFFICIAL_BURN_CONFIRMER_ROLE) {
    ...
    uint256 balanceAfterOfficialBurn = officialToken.balanceOf(address(this));
    pendingOfficialBurn -= amount;
    pendingRedeemStableProceeds += payoutAmount;
    require(balanceAfterOfficialBurn >= classifiedOfficialBacking(),
"official_balance_deficit");
    ...
}
```

Solution

It is recommended to remove this redundant check to avoid maintaining a misleading pseudo-security validation that does not verify the actual business condition.

Status

Acknowledged

[N7] [Suggestion] Stablecoin Configuration Consistency Is Not Validated

Category: Others

Content

In the LabsWrapperVault contract constructor, the Vault only verifies that `officialFundController_` is a contract address, but does not validate that the stablecoin configured in the `FundController` matches the `stablecoin` configured in the Vault.

If the Vault is deployed with a `FundController` that manages a different stablecoin, subscription operations may continuously fail. Alternatively, if the external implementation performs insufficient validation, assets may be processed under the wrong token assumption, leading to inconsistencies between actual funds and internal accounting records.

Code location: LabsWrapperVault.sol#L242-L243

```
constructor(  
    ...  
) {  
    ...  
    wrappedToken = ILabsWrappedToken(wrappedToken_);  
    stablecoin = IERC20(stablecoin_);  
    ...  
}
```

Solution

The constructor should enforce an explicit consistency check to ensure that the `stablecoin` configured in `FundController` is identical to the `stablecoin` configured in the Vault.

Status

Acknowledged

[N8] [Low] Global refId Can Be Preemptively Occupied Across Different Workflows

Category: Denial of Service Vulnerability

Content

In the LabsWrapperVault contract, the `requestWithdrawal` function allows any caller to provide any non-zero `requestId`, and the value is permanently occupied through the global `processedRefs` mapping. The same global reference tracking mechanism is shared across subscription, official minting, redemption, and settlement workflows. An attacker can monitor pending transactions in the mempool and observe a legitimate `refId` before it is included on-chain. The attacker can then front-run the transaction by calling the publicly accessible `requestWithdrawal` function with the same reference ID, consuming the `refId` through an event-only operation. As a result, the legitimate business transaction will revert due to `ref_used`.

This creates a cross-workflow denial-of-service risk where valid operations can be blocked by unauthorized reference ID occupation. However, exploitation requires the attacker to continuously spend gas to compete for transaction ordering.

Code location: LabsWrapperVault.sol#L658

```
function requestWithdrawal(  
    ...  
    bytes32 requestId  
) external nonReentrant whenNotPaused {  
    ...  
    _consumeRef(requestId);  
    ...  
}
```

Solution

It is recommended to separate reference namespaces between different business flows. For example,

`requestWithdrawal` should use a dedicated order ID mechanism rather than sharing the same global

`processedRefs` space with unrelated workflows.

Status

Acknowledged

[N9] [Suggestion] Withdrawal Requests Only Record Unverified User Input

Category: Design Logic Audit

Content

In the LabsWrapperVault contract, the `requestWithdrawal` function only checks that the input fields are non-zero before emitting the `WithdrawalRequested` event. It does not verify whether the user actually owns the assets corresponding to `assetId`, whether `sourceRef` is valid, or whether the specified assets are eligible for withdrawal. As a result, any address can construct arbitrary withdrawal request events. If the off-chain system directly treats these events as withdrawal authorization instructions, an attacker could potentially induce unauthorized withdrawals. The current security model therefore relies entirely on the off-chain system performing additional verification before executing any payout.

Code location: LabsWrapperVault.sol#L659

```
function requestWithdrawal(
    ...
) external nonReentrant whenNotPaused {
    ...
    emit WithdrawalRequested(requestId, msg.sender, receiver, assetId, assets,
    sourceType, sourceRef);
}
```

Solution

The `WithdrawalRequested` event should be explicitly treated as an untrusted request record rather than an authorization credential. The off-chain execution system must independently verify the user identity, asset ownership or balance, source records, order status, and idempotency before processing any withdrawal. If stronger on-chain guarantees are required, the protocol should introduce verifiable balance checks or require a withdrawal authorization signed by the user or an authorized backend service.

Status

Acknowledged

[N10] [Low] Critical Asset Flows Depend on a Centralized Trust Chain

Category: Authority Control Vulnerability Audit

Content

In LabsWrappedToken and LabsWrapperVault, `DEFAULT_ADMIN_ROLE` has control over critical roles, while the Vault relies on `officialFundController`, `RECEIPT_MANAGER_ROLE`, operational roles, and off-chain settlement information to complete asset registration, minting, redemption, and payout processes.

Although `registerOfficialMintSettlement` enforces an upper limit based on the actual balance of official tokens, this mechanism cannot replace the fundamental trust placed in privileged roles and external settlement systems.

This issue represents an architectural trust assumption of the RWA design rather than an unauthorized external attack vector. However, compromise of privileged keys, incorrect permission configuration, or inaccurate off-chain

settlement data could directly affect user asset ownership records and overall system availability.

Solution

It is recommended to manage administrators and critical operational roles through multisignature or threshold-control mechanisms. Additional safeguards should include strict role separation, per-transaction and periodic spending limits, delayed execution mechanisms, and comprehensive audit logging.

Status

Acknowledged; After discussion with the project team, the team confirmed that this is a necessary design choice for an RWA protocol. The overall operational workflow of the protocol will be managed by centralized institutions.

[N11] [Low] Official Mint Rejection Lacks a User Refund Path

Category: Design Logic Audit

Content

In the `LabsWrapperVault` contract, the `registerOfficialMintRejection` function is responsible for recording the rejection result after an official `requestMint` request is rejected and accounting for the returned stablecoins back into the contract's internal pending balance.

According to the current implementation, the function only decreases `pendingOfficialMintStable` and adds the same amount back into `pendingStableDeposits`, effectively treating the rejected funds as returning to the pool of "pending mintable deposits." This means privileged roles can later use these user funds to initiate another official mint request.

However, the protocol does not provide an active refund path tied to the original `payer` or `recipient`. The rejection result is also not mapped back to the corresponding deposit order for terminal settlement. As a result, if a user does not wish to retry the minting process, their funds may remain locked inside the contract, with the decision of whether to resubmit the mint request controlled entirely by operators rather than by the user.

Code location: `LabsWrapperVault.sol#L659`

```
function registerOfficialMintRejection(  
    uint256 stableAmount,  
    uint256 officialRequestId,  
    bytes32 refId,  
    bytes32 evidenceHash  
) external nonReentrant whenNotPaused onlyRole(RECEIPT_MANAGER_ROLE) {  
    ...  
}
```

Solution

For official mint rejection scenarios, the protocol should introduce explicit terminal state handling for the original deposit order.

The preferred approach is to maintain a relationship between `officialRequestId` and the corresponding deposit order. After rejection, the protocol should allow a trusted role or the user to refund the stablecoins back to the original `payer`, while updating the order status to `Rejected` or `Refunded`. This prevents rejected funds from being returned to `pendingStableDeposits` and later reused for unrelated mint requests.

If automatic retry of rejected mint requests is an intended business flow, the protocol should introduce explicit user authorization or a cancellation mechanism. Additional state fields and events should also be added to clearly separate the “refund” path from the “resubmission” path, ensuring both workflows remain transparent and auditable.

Status

Acknowledged; After discussion with the project team, the team confirmed that although `FundController` provides a rejection mechanism, in the actual business process it will not reject requests submitted by the Vault.

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
OX002608040001	SlowMist Security Team	2026.08.04 - 2026.08.04	Low Risk

Summary conclusion: The SlowMist security team uses a manual and the SlowMist team's analysis tool to audit the project. During the audit, we found 1 medium-risk, 5 low-risk, and 5 suggestions. All the findings were acknowledged. The code was not deployed to the mainnet.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>